Business Requirements Document (BRD)

Project Title

- Orders and Customer Data Foundation using Delta Lake / Delta Live Tables (DLT)

Prepared by

- Senior Business Analyst

Date

- [Insert date]

Version

- 1.0

1. Executive Summary

- Purpose: Create a robust, auditable, and production-ready ingestion and transformation pipeline to read customer and order CSV source files, cleanse and enrich data, maintain historical customer/order relationships using Slowly Changing Dimension (SCD) Type 2, and provide aggregate spend information per customer per date. Implementation will use Delta Lake tables and will be orchestrated using Databricks Delta Live Tables (DLT).
- Business value: Provides consolidated customer-order view with historical tracking and ready-to-use aggregated metrics for analytics and reporting (e.g., spend trends, customer lifecycle analysis). Improves data quality, lineage, and reproducibility.

2. Background and Business Drivers

- Current state: Source CSV files (customer and order) exist on a storage volume. There is no standardized pipeline to load to Delta tables, enforce data quality, compute derived fields, manage historical changes, and produce aggregated outputs.
- Drivers:
- Need for a canonical ordersummary dataset for analytics and historical analysis.
- Need for accurate aggregated spend by customer/date for reporting and downstream consumption.
- Requirement for data quality (null handling, deduplication).
- Use of Delta Live Tables for simplified, declarative pipeline management, monitoring, and enforcement.

3. Scope

- In scope:
- Read customer and order CSV data from defined storage paths.
- Persist data into Delta tables named customer and order.
- Apply schema definitions for both source tables (fields listed in section 4).

- Compute TotalAmount in orders as PricePerUnit * Qty.
- Remove records that contain nulls or the literal string "Null" for any field; remove duplicate rows.
- Create an ordersummary Delta table in the specified catalog/schema if it does not exist.
- Join customer and order on CustId to produce ordersummary.
- Implement ordersummary as an SCD Type 2 table: track history of customer attribute changes (mark previous records inactive, insert new active records, maintain StartDate and EndDate).
- Create a customeraggregatespend table to store aggregated TotalAmount by Name and Date.
- Aggregate TotalAmount by Name and Date from ordersummary and load into customeraggregatespend.
- Implement the pipeline using Delta Live Tables.
- Out of scope:
- Additional enrichment from external systems beyond customer and order CSVs.
- Complex data validation rules beyond null/"Null" removal and duplicate elimination (unless specified later).
- Downstream reporting implementation (BI dashboards) — only prepared datasets will be delivered.
- Security/access configuration beyond basic catalog/schema/table creation (to be coordinated with platform/infra teams).

4. Source and Target Data Details (Schemas, Paths)

- Source locations:
- customer CSV source path: /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata
- order CSV source path: /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata

- Source table schemas:
- customer
- CustId (unique identifier for customer)
- Name (customer full name or display name)
- EmailId (customer email address)
- Region (customer region)
- order
- OrderId (unique identifier for order)
- ItemName (name of the item purchased)
- PricePerUnit (numeric price per unit)
- Qty (quantity ordered; numeric)
- Date (order date)
- CustId (foreign key to customer.CustId)

- Derived field:

- order.TotalAmount = PricePerUnit * Qty
- Data type: numeric (appropriate precision/scale, e.g., Decimal(18,2) or Double depending on business requirements)

- Target Delta catalog/schema/tables:
- Catalog: gen_ai_poc_databrickscoe
- Schema: sdlc_wizard
- customer (Delta table)
- order (Delta table)
- ordersummary (Delta table implementing SCD Type 2)
- customeraggregatespend (Delta table)

- ordersummary schema (business-level fields):
- CustId
- Name
- EmailId
- Region
- OrderId
- ItemName
- PricePerUnit
- Qty
- Date
- TotalAmount (derived)
- SCD management columns (required for Type 2):
- StartDate (date/timestamp when this version became effective)
- EndDate (date/timestamp when this version became obsolete; null or a sentinel for active)
- IsActive (boolean or string: e.g., 'Active'/'Inactive' or 1/0)
- VersionId or HashKey (optional — to identify distinct versions)
- EffectiveFromTimestamp / EffectiveToTimestamp (optional high-precision)
- Audit metadata (recommended):
- CreatedAt
- CreatedBy
- UpdatedAt
- UpdatedBy
- SourceFileName or LoadBatchId

- customeraggregatespend schema:

- Name
- TotalAmount (aggregated, numeric)
- Date

5. Functional Requirements

FR-1: Ingestion

- The system must read CSV data from the provided source paths and load them into Delta tables named customer and order.
- Ingestion must capture file-level metadata (file name, ingestion timestamp, batch id) for traceability (recommended).

FR-2: Schema Enforcement

- The customer and order Delta tables must conform to the defined schemas.
- Data types must be explicitly defined; if types are unknown, coordinate with data owners to decide (PricePerUnit: decimal; Qty: integer; Date: date/timestamp; CustId/OrderId: string or integer based on upstream systems).

FR-3: Calculated Column

- The order table must include a TotalAmount column computed as PricePerUnit * Qty (data type: numeric). Calculation must handle nulls robustly (if either PricePerUnit or Qty is null or "Null", the record should be removed per data quality rules; or TotalAmount should be computed only for valid numeric rows).

FR-4: Data Quality

- Remove records that contain null values or the literal string "Null" in any field.
- Remove exact duplicate rows (all columns identical) from both customer and order tables.
- Log the number of rows removed and retain a quarantine location for rejected records for later inspection.

FR-5: Ordersummary Creation and Join

- Create the ordersummary table in catalog gen_ai_poc_databrickscoe, schema sdlc_wizard if it does not already exist.
- Join customer and order using CustId. The resulting ordersummary must include customer attributes joined to each order, plus TotalAmount.
- Ensure inner vs left join semantics is decided: default should be inner join to only include orders with matching customer record; if business requires orders without customer to be preserved, use left join and mark missing customer attributes as null (but then their presence may violate data quality rule to drop nulls — clarify with stakeholders).

FR-6: SCD Type 2 Behavior for ordersummary

- Implement SCD Type 2 on customer-level attributes within ordersummary such that:

- When a customer's attributes change (e.g., Name, EmailId, Region), prior records for that customer must be marked Inactive (IsActive = false or 'Inactive') and their EndDate set to the time of change.
- A new record reflecting latest attributes must be inserted and marked Active (IsActive = true or 'Active') with StartDate set to the change time and EndDate null (or a high sentinel date).
- Order-level events (OrderId, ItemName, etc.) should remain associated with the relevant customer version according to business rules: typically, orders are tied to the customer version effective at the order Date. Confirm business rule: on customer attribute change, should historical orders remain associated with the prior customer version (common SCD2 approach) — default is yes.
- SCD implementation must be incremental: detect changes in customer attributes and update ordersummary accordingly without full reloads where possible.
- Maintain audit metadata and versioning to support traceability.

FR-7: Update Behavior Trigger

- Ordersummary SCD Type 2 must be updated when there are changes in the customer table. Changes include new customers, attribute updates, or deletions.
- For deletions, decide on strategy: mark records inactive with an EndDate and IsActive=false; optionally insert a tombstone record.

FR-8: Aggregation

- Create customeraggregatespend table in gen_ai_poc_databrickscoe.sdlc_wizard if it does not exist with columns Name, TotalAmount, Date.
- Aggregate TotalAmount from ordersummary grouped by Name and Date using SUM(TotalAmount) as TotalAmount.
- Load aggregated results into customeraggregatespend. Define whether this is a full rebuild or incremental upsert. Prefer incremental upsert using Date as partition key and Name as business key when possible.

FR-9: Implementation Technology

- Implement pipeline with Delta Live Tables to enforce declarative transformations, data quality constraints, monitoring, and lineage.
- Use DLT constructs for defining input datasets, transformations, expectation rules (e.g., no nulls, uniqueness), and output tables.

6. Non-Functional Requirements

NFR-1: Performance and Scalability

- Pipeline must scale to expected volume (define expected record rates & sizes in follow-up). Use partitioning (e.g., Date) on large tables to improve query performance.
- Queries for ordersummary and customeraggregatespend must return results within acceptable SLAs for analytic workloads (to be agreed).

NFR-2: Reliability and Idempotency

- Pipeline must be idempotent: re-running ingestion should not create duplicates, and SCD updates should be deterministic.
- Ensure transactional writes via Delta to maintain ACID semantics.

NFR-3: Data Quality & Monitoring

- Implement DLT expectations to enforce rules:
- No nulls or literal "Null" in required fields.
- Unique constraints on CustId (customer) and OrderId (order) if applicable.
- Provide monitoring/alerting on expectation violations and counts of dropped/quarantined records.

NFR-4: Security & Access

- Tables created in catalog gen_ai_poc_databrickscoe.sdlc_wizard must follow organization access controls. Define roles for data engineers, analysts, and auditors.
- Ensure sensitive fields like EmailId are handled per privacy policy (masking or restricted access) if required.

NFR-5: Maintainability & Extensibility

- Pipeline code organized and documented; parameterize file paths, catalogs, schemas, and thresholds.
- Include automated tests for transformations and SCD logic.

NFR-6: Auditability and Lineage

- Maintain audit columns (CreatedAt, UpdatedAt, LoadBatchId, SourceFile) and use Delta's transaction log for lineage and rollback if required.

7. Data Quality Rules (detailed)

- DQ-1: Drop any record where any column equals null or the string "Null".
- DQ-2: Remove duplicate records (exact duplicates) across all columns of the record.
- DQ-3: PricePerUnit and Qty must be numeric. Non-numeric values must be quarantined.
- DQ-4: Date must be parsable into a date/timestamp. Invalid dates must be quarantined.
- DQ-5: For customer: CustId must be present and unique (if duplicates occur, treat per business rule — reject to quarantine or apply dedupe logic).
- DQ-6: For order: OrderId must be present and unique (same dedupe decision).
- DQ-7: All dropped/quarantined records must be logged with reason, source file, and timestamp.

8. Business Rules and Assumptions

- Default Join Behavior: Unless stakeholders require otherwise, use inner join for ordersummary to only include orders with a matching customer record. If requirement changes, support left join and a strategy for handling missing customer attributes.

- SCD association rule: Orders are associated with the customer version active at the order Date. On customer attribute updates occurring after an order Date, the order remains associated with the previous customer version unless the business explicitly wants to re-associate historical orders.
- Date granularity for aggregation: The Date column is the aggregation key; clarify whether aggregation uses order date (Date) or a business date. Timezone normalization to be applied if incoming dates vary in timezones.
- Name used as aggregation key in customeraggregatespend: using Name as key assumes Name is unique enough to identify customers. Prefer using CustId for same-customer grouping; include Name in final dataset for readability. Recommend using CustId + Date for aggregation key, plus Name as display column to avoid grouping unrelated identical Names.
- Data types and precision need stakeholder confirmation (numeric precision for TotalAmount).

9. Functional Design / Process Flow (High-level)

- Step 1: Ingest CSVs
- DLT reads customer and order CSVs from given volume paths and writes to Delta tables customer and order with schema enforcement.
- Tag ingestion with batch metadata.
- Step 2: Apply DQ and transformations
- Apply DQ expectations to drop or quarantine null/"Null" and invalid types.
- Deduplicate both tables.
- Compute TotalAmount in the order table (PricePerUnit * Qty).
- Step 3: Create ordersummary
- Create ordersummary if not exists in target catalog/schema.
- Join customer and order on CustId (inner join by default) and generate full rowset with SCD management columns.
- Step 4: SCD Type 2 Management
- For new or changed customer attributes, close old SCD records (set EndDate, IsActive=false) and insert new active SCD records with StartDate and IsActive=true.
- Ensure orders are assigned to the correct version by effective dates.
- Step 5: Aggregation
- Aggregate SUM(TotalAmount) by Name and Date (or by CustId and Date recommended).
- Write results to customeraggregatespend (create if not exists).
- Step 6: Orchestration and Monitoring
- Implement entire pipeline in Delta Live Tables with expectations and monitoring dashboards. Set schedule as required.

10. Data Model and Example Records

- Sample customer record:
- CustId: 1001

- Name: "Acme Corp"
- EmailId: "contact@acme.com"
- Region: "EMEA"
- Sample order record:
- OrderId: 2001
- ItemName: "Widget"
- PricePerUnit: 10.50
- Qty: 4
- Date: 2025-03-01
- CustId: 1001
- TotalAmount: 42.00
- Sample ordersummary SCD Type 2 record:
- CustId: 1001
- Name: "Acme Corporation" (old)
- EmailId: "old@acme.com"
- Region: "EMEA"
- OrderId: 2001
- ItemName: "Widget"
- PricePerUnit: 10.50
- Qty: 4
- Date: 2025-03-01
- TotalAmount: 42.00
- StartDate: 2023-01-01
- EndDate: 2024-05-01
- IsActive: false
- CreatedAt, LoadBatchId, SourceFileName, etc.

11. Implementation Considerations and Recommendations

- Use Delta Live Tables for:
- Declarative pipeline definitions.
- Built-in expectations for data quality and monitoring.
- Automated handling of streaming vs batch (DLT supports both).
- Use CustId as primary business key for join and SCD management; avoid aggregating solely on Name to prevent mis-grouping similar names.
- Implement partitioning strategy:
- For order and ordersummary: partition by Date (e.g., yyyy-MM-dd or year/month) for performance.

- For customeraggregatespend: partition by Date.
- Indexing/Optimization: use Z-ordering on frequently filtered columns (e.g., CustId, Date) where supported.
- Quarantine strategy:
- Store failed records in a separate Delta table or storage path with reason codes.
- Build a small monitoring report for DQ failures per run.
- SCD Performance:
- Use merge (MERGE INTO) capability of Delta for efficient upserts when implementing SCD Type 2.
- Consider using hash of customer attributes to quickly detect changes.
- Testing:
- Unit tests for transformation logic, including edge cases (nulls, string "Null", non-numeric numbers).
- Regression tests for SCD transitions (multiple updates).
- Deployment:
- Parameterize environment-specific variables (catalog, schema, source path) for dev/test/prod.
- Use CI/CD to deploy pipeline definitions.

12. Acceptance Criteria

- Data ingestion:
- Customer and order Delta tables are populated from the specified source paths.
- Loaded data conforms to the expected schema and data types.
- Transformations:
- TotalAmount present and correctly computed for all valid order records.
- Records with nulls or "Null" and duplicates removed; quarantined records available for inspection.
- Ordersummary:
- ordersummary table exists in gen_ai_poc_databrickscoe.sdlc_wizard and contains joined customer-order records.
- SCD Type 2 behavior correctly tracks changes to customer attributes (old records inactive with EndDate set, new records active with StartDate set).
- Orders remain associated with the correct customer version according to defined business rule.
- Aggregates:
- customeraggregatespend table exists and contains SUM(TotalAmount) grouped by Name and Date (or by CustId+Date if implemented).
- Implementation:
- All steps are implemented as Delta Live Tables with expectations in place.
- Monitoring/alerts configured for expectation failures and pipeline errors.
- Documentation and Tests:
- Pipeline code documented and unit/integration tests exist and pass.

- Operational runbook documented, including recovery steps and contact points.

13. Risks and Mitigations

- Risk: Ambiguity on join semantics (inner vs left) may lead to unexpected data loss.
- Mitigation: Confirm requirement with stakeholders; default to inner join and document consequences.
- Risk: Aggregation by Name may group distinct customers with identical names.
- Mitigation: Use CustId for aggregation keys; include Name as display attribute; seek stakeholder sign-off.
- Risk: Inconsistent date formats/timezones in source data.
- Mitigation: Normalize date formats during ingestion; apply timezone rules; quarantine unparsable dates.
- Risk: Large historical changes causing heavy SCD updates and performance issues.
- Mitigation: Implement incremental SCD logic and efficient MERGE patterns; run large backfills during off-peak windows.
- Risk: Sensitive data exposure (EmailId).
- Mitigation: Apply column-level access controls and masking as required by privacy policy.

14. Open Questions / Decisions Required

- Should the ordersummary join be inner or left? If left, how should orders without customers be handled?
- For aggregation: should grouping use CustId+Date (recommended) rather than Name+Date? Confirm.
- Exact data types and precisions for PricePerUnit, TotalAmount, and Qty.
- Required retention policy for historical SCD records and aggregated tables.
- Expected data volume and acceptable SLAs for pipeline runtimes.
- Policy for handling customer deletions: tombstone vs marking inactive only.
- Quarantine storage location and retention period.

15. Implementation Tasks (High-level)

- Task 1: Create DLT pipeline skeleton and configure source connectors for customer and order CSV paths.
- Task 2: Define input tables with schema enforcement.
- Task 3: Implement DQ expectations and quarantine logic.
- Task 4: Implement computation of TotalAmount and deduplication.
- Task 5: Implement ordersummary creation and SCD Type 2 MERGE/upsert logic.
- Task 6: Implement customeraggregatespend aggregation and upsert logic.
- Task 7: Add audit metadata and logging.

- Task 8: Implement monitoring, alerts, and dashboards for DLT pipeline.
- Task 9: Create unit/integration tests and run validation.
- Task 10: Deploy to production, run verification, and handover documentation/runbook.

16. Roles and Responsibilities

- Data Engineering:
- Build the DLT pipeline, implement transformations, SCD logic, and aggregation.
- Configure Delta tables and storage access.
- Data Architect:
- Approve schema and partitioning strategy; ensure alignment with platform standards.
- Data Steward / Business Owner:
- Validate business rules, join semantics, aggregation keys, and acceptance criteria.
- Platform/Infra:
- Provide storage, compute, catalog access, and security roles.
- QA / Test:
- Execute tests and validate data quality and SCD behavior.
- Operations:
- Monitor pipeline post-deployment, manage incidents.

17. Deliverables

- Delta Live Tables pipeline code and configuration.
- Delta tables: customer, order, ordersummary, customeraggregatespend in specified catalog/schema.
- DQ/quarantine tables and monitoring dashboards.
- Unit and integration test artifacts.
- Operational runbook and data dictionary documenting schemas, fields, and business logic.

Appendices

A. Glossary

- Delta Lake, Delta Live Tables (DLT), SCD (Slowly Changing Dimension), DQ (Data Quality), Quarantine (rejected records store).

B. References

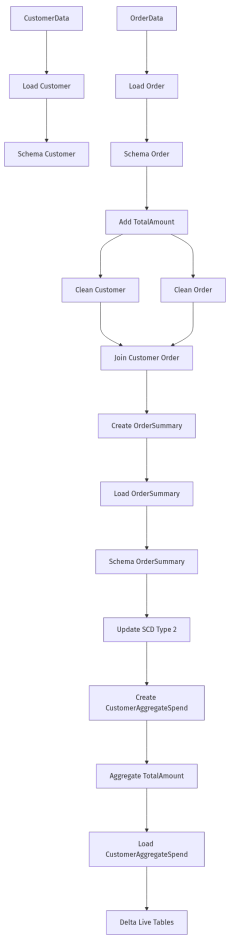
- Organization data governance policies (to be attached).
- Databricks Delta Live Tables documentation (implementation guidelines).
- Delta MERGE statements and examples for SCD Type 2 patterns.

End of Document

Note: Several design choices require stakeholder confirmation (join type, aggregation key, data types, and SCD association of orders). Recommend a short requirements validation workshop with business stakeholders and data owners to finalize these open items prior to development.

Data Flow Diagram (DFD)

Data Flow Diagram (DFD)




```mermaid

graph TD;

CustomerData[CustomerData] ---> LoadCustomer[Load Customer];

OrderData[OrderData] ---> LoadOrder[Load Order];

LoadCustomer[Load Customer] ---> SchemaCustomer[Schema Customer];

LoadOrder[Load Order] ---> SchemaOrder[Schema Order];
```

```
SchemaOrder[Schema Order] ---> AddTotalAmount[Add TotalAmount];

AddTotalAmount[Add TotalAmount] ---> CleanCustomer[Clean Customer];

AddTotalAmount[Add TotalAmount] ---> CleanOrder[Clean Order];

CleanCustomer[Clean Customer] ---> JoinCustomerOrder[Join Customer Order];

CleanOrder[Clean Order] ---> JoinCustomerOrder[Join Customer Order];

JoinCustomerOrder[Join Customer Order] ---> CreateOrderSummary[Create OrderSummary];

CreateOrderSummary[Create OrderSummary] ---> LoadOrderSummary[Load OrderSummary];

LoadOrderSummary[Load OrderSummary] ---> SchemaOrderSummary[Schema OrderSummary];

SchemaOrderSummary[Schema OrderSummary] ---> UpdateSCDType2[Update SCD Type 2];

UpdateSCDType2[Update SCD Type 2] ---> CreateCustomerAggregateSpend[Create
CustomerAggregateSpend];

CreateCustomerAggregateSpend[Create CustomerAggregateSpend] --->
AggregateTotalAmount[Aggregate TotalAmount];

AggregateTotalAmount[Aggregate TotalAmount] ---> LoadCustomerAggregateSpend[Load
CustomerAggregateSpend];

LoadCustomerAggregateSpend[Load CustomerAggregateSpend] ---> DeltaLiveTables[Delta Live
Tables];

```